ZipperGen Workflow Development and Deployment Manual

Concepts, commands, configuration, runtime, and operations

The ZipperGen Authors

Manual edition: 25 August 2026

Abstract

This manual is what you need after the tutorial. It covers the project's files, the workflow language, inspection and validation, running and testing, models and connectors, deployment and operations, and what to do when something is wrong.

It assumes you can use a terminal and an editor. It does not assume you are a Python or operations engineer, and it does not assume you use a coding agent, though it says where one helps.

Start with the tutorial

`docs/first-workflow.pdf` builds one small application end to end in about seven pages. This manual explains the parts it passes over. If you have not read it, read it first. Most of what follows will then be lookup rather than learning.

The promise

The workflow is always ordinary Python. A coding agent may write or change it, but ZipperGen does the deterministic work: loading it, projecting it for every participant, checking that the result is well formed, running it durably, and deploying it. Opaque model output never becomes a deployed application without passing through visible code you can read.

Contents

1 The project

A ZipperGen project is a directory. `zg init` creates one:

File	In git	Holds
`specification.md`	yes	what the workflow should do, in prose
`workflow.py`	yes	the coordination, as Python
`zippergen.toml`	yes	every non-secret choice: models, assistants, connectors, deployment answers, entry point
`AGENTS.md`	yes	points a coding agent at `zg skill`
`CLAUDE.md`	yes	points Claude Code at `AGENTS.md`
`.zippergen/`	no	this checkout's generated identity and stable private workspace name; ignores itself
`ZIPPERGEN_HOME`	no	credentials, runs, deployments

The split that matters is between the committed rows and the last two. Everything committed travels with the project. A colleague who clones the repository gets the workflow, the specification and the configuration, and supplies their own keys.

`.zippergen/` holds the generated identity that keys this checkout's private state and the stable name of that state directory. Neither may travel with a clone: two checkouts sharing them would share one credential store. So they are not in `zippergen.toml`, they are not committed, and the directory contains a `.gitignore` of its own so no project needs an entry for it. A clone has no identity at all and is keyed by its own path. `zippergen init` does not change that: it returns immediately when a manifest already exists, and an identity is minted only when a project is created. A clone therefore never inherits the original's private state, because it never learns the original's identity. Move the whole working directory and both local values move with it, preserving the workspace and deployment address. There is no rule to remember. The private workspace name keeps the directory prefix from when it was first recorded; after a move that prefix is historical, not stale state to delete.

1.1 `zippergen.toml`

```
schema_version = 2
name = "email-approval"
specification_file = "specification.md"
workflow_entry = "workflow.py:email_approval"

[providers.connections."openai-main"]
kind = "openai"

[providers.connections."approval-bot"]
kind = "telegram"

[models.configurations."openai-gpt-4o"]
connection = "openai-main"
model = "gpt-4o"

[models.assignments]
default = "mock"

[models.assignments.lifelines]
Writer = "openai-gpt-4o"
```

```
21
22 [connectors.configurations."approval-chat"]
23 connection = "approval-bot"
24 kind = "telegram"
25 chat_id = "12345678"
26
27 [connectors.assignments.lifelines]
28 Mailbox = "approval-chat"
29
30 [configuration]
31 recipient = "intake@example.org"
32 send_mode = "draft"
```

Everything here except `schema_version` is a choice somebody made. The stamp lets an older ZipperGen refuse a file whose newer fields it would erase; the generated identity remains local because it must not travel with a clone.

It is an ordinary file. Edit it in an editor, or let a coding agent edit it. Nothing in ZipperGen owns it. `workflow_entry` is why most commands do not need you to name the workflow, and

```
zg workflow select workflow.py:email_approval
```

records it without editing the file. With no argument it prints the current one. A project with a single workflow needs no entry at all.

Named configurations exist so several participants can share one and it changes in one place. `configurations` says what a thing is, and `assignments` says who uses it.

1.2 What stays on one machine

API keys	environment variables, or `ZIPPERGEN_HOME`
Telegram bot token	typed once, never an argument
Google credentials	`zg provider authorize` / `accept`
Local model endpoints	site configuration
Runs and deployments	`ZIPPERGEN_HOME`

2 Writing workflows

A workflow is a Python function decorated with `@workflow`. Its body is read, rewritten, and turned into an intermediate representation, so it looks like Python and reads like a protocol, but it is not executed line by line as written.

2.1 Participants and messages

```
1 Mailbox = Lifeline("Mailbox")
2 Writer = Lifeline("Writer")
3
4 Mailbox(message) >> Writer(message)
```

The left side sends, the right receives. The names in brackets say which variables carry the value at each end. They need not match.

A self-send (`A(x) >> A(y)`) is a local rename, not a message.

2.2 Actions

An action is work one participant does.

`@llm`	a model call with a prompt and declared outputs
`@pure`	ordinary Python, output name and type from the return annotation
`@effect`	Python that may touch the outside world; resolved outside the durable write transaction
`@human`	stops and asks a person: `confirm`, `select`, `input`
`@assistant`	runs a coding agent against a repository
`@planner`	asks a model to generate a sub-workflow, validated before it runs

```
@llm(
    system="You write short, friendly replies.",
    user="Reply to this message:\n\n{message}",
    parse="text",
    outputs=[("draft", str)],
)
def draft_reply(message: str): ...

Writer: draft = draft_reply(message)
```

For `@pure`, `@effect`, and `@assistant`, `visible=False` is a persistence choice rather than a display filter: the action's own trace events are never written. It does not hide decisions or control sends and receives around that action. Use it only for intentionally unrecorded operational work, not to conceal meaningful protocol behavior.

Model failures are part of the action declaration, not provider-specific runtime configuration. A finite integer counts retries after the first call; `"forever"` waits until success or cancellation. An optional fallback is validated against the declared outputs when the action is defined:

```
@llm(
    system="Classify the request.",
    user="{message}",
    parse="bool",
    outputs=[("urgent", bool)],
    retries=3,
    fallback=False,
)
def classify(message: str): ...
```

Connection failures, timeouts, rate limits, server failures, and invalid model answers use this one retry budget. Authentication failures and bad model names fail immediately (or take the fallback). `Retry-After` is respected; otherwise the delay is 2, 4, 8, ... seconds, capped at 30. Stopping the run interrupts the wait and never takes the fallback.

Several actions by one participant can be grouped:

```
with LLM1:
    agreed = check_agreement(verdict, other_verdict)
    trials = inc_trials(trials)
```

2.3 Decisions

```
if approved @ Mailbox:
    Mailbox: result = sent(draft)
else:
    Mailbox: result = discarded()
```

The `@ Mailbox` names the participant who evaluates the condition. It is required, and it is the most important annotation in the language: it decides who broadcasts the outcome and who merely receives it.

Loops work the same way:

```
while (not agreed and trials < MAX_ROUNDS) @ LLM1:
    ...
```

Parenthesise compound conditions

`@` binds more tightly than `not`, `and` and `or`. Write `if (not agreed) @ LLM1:`, not `if not agreed @ LLM1:`.

2.4 Parallel regions and co-regions

Independent branches that may interleave:

```
with parallel:
    with branch:
        Orchestrator(candidate) >> TestRunner(candidate)
        TestRunner: (test_status,) = run_tests(candidate)
        TestRunner(test_status) >> Committer(test_status)
    with branch:
        Orchestrator(candidate) >> Security(candidate)
        Security: (security_status,) = scan_security(candidate)
        Security(security_status) >> Committer(security_status)
```

A co-region is weaker: messages that may arrive in any order.

```
with coregion:
    Analyst_A(a) >> Collector(a_report)
    Analyst_B(b) >> Collector(b_report)
```

2.5 Inputs and outputs

```
def email_approval() -> int:
    ...
    return handled @ Mailbox
```

An input belongs to the participant that supplies it. The return says who holds the result. `zg validate` lists all declared workflow inputs. If an interactive run still needs any values, it introduces them under a `Workflow inputs` heading and shows declared defaults.

3 Inspecting and validating

3.1 `zg validate`

```
zg validate
```

Loads the module, projects the protocol for every participant, checks the deployment declaration, and renders the result back. If it passes, the workflow is well formed and the deadlock-freedom guarantee applies to it.

This is the check to run after every change. It is fast and needs nothing configured.

3.2 `zg show`

`zg show`	the global protocol
`zg show --communications`	messages only, no action bodies
`zg show --agent NAME`	one participant's local program
`zg show --detail full`	everything, including action definitions
`zg show --format json`	structured, for tooling

`--agent` is the interesting one. It shows what a participant actually runs after projection, which is not what you wrote:

```
@role('Writer')
def email_approval__Writer() -> None:
    while recv_decision('Mailbox'):
        message = recv('Mailbox')
        draft = draft_reply(message)
        send('Mailbox', draft)
```

The `Writer` is told each round whether to continue, because it has work inside the loop. It has no `if`, because it takes no part in the `Mailbox`'s decision. A participant's local program mentions only what concerns it, which is why it cannot wait for a message the decider chose not to send.

Use this when a workflow behaves in a way you did not expect. The global view says what you wrote. The local view says what will run.

4 Running and testing

4.1 One verb

`zg run`	execute once, nothing is kept
`zg run --durable`	record every step, so it can be resumed
`zg run --resume`	continue the most recent unfinished run

A plain run leaves no durable record to resume or inspect. That is deliberate: throwaway runs should not accumulate. When you want to come back to a run, to resume it, inspect it, or answer a human action later, use `--durable`.

Missing inputs are asked for in a terminal. In a script, supply them:

```
zg run --input message="Could we move our meeting to Thursday?" --yes
```

4.2 One configuration pattern

Provider connections, models, coding assistants, and connectors use one small grammar:

```
zg provider configure NAME KIND
zg TYPE configure NAME ...
zg TYPE assign TARGET NAME
zg TYPE check [NAME]
zg TYPE remove NAME
```

A connector target is either a service requirement the workflow declares by name or a participant whose actions ask a human; the workflow says which, so `assign` covers both and there is nothing to choose. The provider connection is the named access path to an external provider: it owns the private credential and any machine-specific endpoint. A model selects a model through one connection; a connector selects a chat, mailbox, sheet, or other resource through one connection. In the examples, `approval-bot` is a Telegram provider connection and `approval-chat` is a connector configuration using it. Bare `zg provider`, `zg model`, `zg assistant`, and `zg connector` show each family. `zg config` shows the complete effective result.

In an interactive terminal, required values may be left out. ZipperGen asks for them and shows known targets and configurations when useful. Thus `zg model configure`, `zg assistant configure`, and `zg connector configure` are guided commands. A script or coding agent must pass the required values explicitly, so it cannot wait for an unseen question. Guided model setup asks for a provider connection and model separately. The Python API is always explicit and never prompts.

4.3 Choosing a model

For a durable project choice, create a named configuration and assign it:

```
zg provider configure openai-main openai
zg provider set-credential openai-main
zg model configure writer openai-main gpt-4o-mini
zg model assign Writer writer
zg model
```

The credential command prompts without echo. The key is saved in the owner-only `ZIPPERGEN_HOME/workspaces/<project>/development.secrets.json` file on this computer. It is never written to `zippergen.toml`. One provider key may serve several named model configurations. You may instead set the normal environment variable. Several model configurations can share the same provider connection, or use different connections when they need different keys.

`--llm mock`	a placeholder for every action, no key needed
`--llm openai:gpt-4o-mini`	a real provider
`--llm scripted:replies.json`	recorded answers, see below
`--llm-for Writer=openai:gpt-4o`	override one participant or action

Without `--llm`, the assignments in `zippergen.toml` apply. The same resolution is used for plain runs, durable runs and deployments. `--llm` is a global one-command override, so it replaces participant and action assignments. Use `--llm-for` for a narrower temporary override. A deployment stores the resolved model specs in its profile; deploy again after changing the project assignments.

`zg model` displays routing without contacting a provider. `zg model check` verifies readiness and makes a small real provider request without printing the credential. Use `zg model unassign TARGET` before removing a configuration that is still referenced.

4.4 Where an assistant may work

`workspace` names the directory an `@assistant` action reads and, with `access="write"`, edits. Declare it as an **absolute path**.

A relative one is resolved against the root the workflow runs from, and a deployment runs from its immutable bundle under `ZIPPERGEN_HOME` rather than from the project directory. So `../target` names one directory during development and a different, absent one once deployed. That is checked before a deployment is applied:

```
FAIL assistant workspace implement: '../sandbox' resolves to
     ~/.zippergen/apps/demo/sandbox, which does not exist. A deployment runs
     from its immutable bundle, not from the project directory, so a relative
     workspace means something different once deployed. Declare an absolute path
```

An absolute path is not portable, which is correct: a write-capable workspace names one directory on one machine that this action may edit.

4.5 Choosing a coding assistant

An `@assistant` action runs Codex or Claude against a repository. Choose the CLI with the same named configuration and assignment pattern:

```
zg assistant configure coding-agent codex
zg assistant assign Maintainer coding-agent
zg assistant check
```

Assign a participant to cover all of its assistant actions. Use an exact target such as `Maintainer.update_release_notes` when one action needs a different backend. Plain runs, durable runs, and deployments use the same routing. A deployment stores the resolved backends in its profile, so deploy again after changing an assignment.

Backend choice and action permissions are separate. The named configuration chooses Codex or Claude. The `@assistant` declaration still owns its fixed instructions and its `access`, `external_tools`, and `shell` policies. `zg config` shows all of them together. `zg assistant check` verifies that each selected CLI is installed and supports every safety option that ZipperGen relies on. It does not log in. Codex and Claude continue to manage their own authentication. Action input is sent over standard input, backend session persistence is disabled, and assistant subprocesses receive only ordinary process settings plus the selected CLI's authentication variables. They do not inherit model and connector credentials from the workflow process.

4.6 Deterministic testing

`mock` answers every action the same way, so a workflow driven by it takes one path and no other. In a consensus protocol that means immediate agreement. Everything behind a decision is unreachable.

Scripted responses fix that:

```
1 {
```

```
  "LLM1.assess":     {"verdict": "yes", "reason": "unmoved"},
  "LLM2.assess":     {"verdict": "no",  "reason": "unmoved"},
  "LLM1.reconsider": {"verdict": "yes", "reason": "unmoved"},
  "LLM2.reconsider": {"verdict": "no",  "reason": "unmoved"}
}
```

```
zg run --llm scripted:never-agree.json
```

Keys are `action` or `Participant.action`. The more specific one wins. Values follow one rule worth learning:

`{...}`	a constant: every call is answered the same way
`[{...}, {...}]`	a finite sequence: exactly these calls are expected

Running past the end of a list is an error, not a silent repeat. That is the point: if a change makes an action run more often than the script expects, the run fails instead of passing quietly.

Use it for branches, not for prompts

Scripted responses exercise *coordination*: this reviewer disagrees, that loop runs three times, this branch is taken. They say nothing about whether a real model would produce good text. Both matter, and they are different questions.

4.7 Knowing it is still working

A workflow that calls a model or a coding assistant can be busy for minutes with nothing to print, and silence looks exactly like a hang. A foreground run therefore shows what is running and for how long:

```
- Implementer . implement . 3m12s
```

The line is erased when the action finishes, so it never appears in a result. It is drawn on standard error and only into an interactive terminal: redirected output, a deployment log and a test see nothing, and a result can still be piped. It also stops while a person is being asked something, because a prompt is read from the same terminal.

4.8 Durable runs

A durable run records coordination state and completed external-action results in its own SQLite store. An ordinarily interrupted run resumes where it stopped. A crash after an external effect succeeds but before its result is recorded can repeat that effect, so irreversible connectors should use idempotency keys.

```
zg run --durable --llm openai:gpt-4o-mini
# Ctrl-C
zg run status
zg run inspect --agent Writer
zg run trace                # recent protocol events
zg run tasks                # decisions waiting for a person
zg run approve              # answer one
zg run --resume
```

The four inspection verbs are the same ones `zg deploy` offers, aimed at the selected development run instead of the service.

Starting another `zg run --durable` discards the older selected development run and creates fresh state. To abandon the selected run, stop its foreground process with Ctrl-C and use `zg run reset`. ZipperGen permanently deletes its record and SQLite state and clears the current-run selection; add `--archive` only when a recoverable private copy is wanted. Reset does not start another run; use `zg run --durable` when one is wanted. A project has one active execution: a disposable or durable foreground run, or its deployment. Stop the current execution before starting another. Observation commands such as `status`, `inspect`, `trace`, and `tasks` remain available while it runs.

To watch a live durable run from another terminal, use:

```
zg run inspect --watch --agent Writer
```

The display refreshes in place once per second. Ctrl-C closes the display and does not interrupt the run. Use `--interval SECONDS` when a slower refresh is more appropriate.

`running`	executing now
`waiting`	stopped at a human action
`interrupted`	you pressed Ctrl-C
`failed`	a participant raised an error
`done`	finished, with a result

`interrupted` and `failed` can be resumed. `done` cannot.

5 Models, people, and services

5.1 Models

For development, an environment variable is enough:

```
export OPENAI_API_KEY=sk-...
zg run --llm openai:gpt-4o-mini
```

`openai:MODEL`	`OPENAI_API_KEY`
`anthropic:MODEL` (or `claude:`)	`ANTHROPIC_API_KEY`
`mistral:MODEL`	`MISTRAL_API_KEY`
`ollama:MODEL` (or `local:`)	`OLLAMA_BASE_URL`
`mock`	none

To fix the choice in the project rather than the command line, put it in `zippergen.toml` under `models`. The guided command does this and can also collect the key privately:

```
zg provider configure
Provider connection name: openai-main
Available provider kinds: openai, anthropic, mistral, local, scripted, telegram,
    google
Provider kind: openai

zg provider set-credential openai-main
```

```
OpenAI API key (input hidden):

zg model configure
Model configuration name: writer
Available provider connections: openai-main
Provider connection: openai-main
Model name or path: gpt-4o-mini
```

The connection name and the model are project configuration. The key is a site credential. They are deliberately stored in different places, which is also why several model configurations may name one connection and share its key.

5.1.1 A model running on your own machine

A local model is a provider connection of kind `local`, and `ollama` is accepted as a name for the same thing. It has an endpoint instead of a credential, so there is no `set-credential` step:

```
zg provider configure my-ollama local --base-url http://127.0.0.1:11434/v1
zg model configure extractor my-ollama qwen2.5:7b
zg model assign Extractor extractor
```

Guided, it fills the usual endpoint in for you:

```
zg provider configure
Provider connection name: my-ollama
Provider kind: local
OpenAI-compatible base URL [http://127.0.0.1:11434/v1]:
```

The endpoint is site configuration, not project configuration, so `zippergen.toml` records only that `my-ollama` is a local connection. Each machine points it at its own server, and a colleague who clones the repository supplies their own. Any OpenAI-compatible server works; Ollama is simply the common one.

A model configuration that names a local connection also accepts `zg model configure ... --idle-timeout SECONDS`, which unloads the model after that long without a call. It is rejected for any other kind of connection, because only a local server has memory of yours to release.

Any model configuration accepts the settings that describe how the model is invoked, stored beside it in `zippergen.toml`:

```
zg model configure classifier local-main qwen3 \
  --temperature 0.2 --max-tokens 4096 --timeout 120
```

```
[models.configurations."classifier"]
connection = "local-main"
model = "qwen3"
temperature = 0.2
max_tokens = 4096
timeout = 120
```

`--temperature` runs from 0 to 1 and an explicit `@llm(temperature=...)` on one action takes precedence. Zero reduces sampling variation; it is not a determinism guarantee. Some current model families removed sampling parameters, so configuration checks reject an explicit unsupported value rather than silently dropping it. `--max-tokens` caps one response and `--timeout` is how long one call may take.

These are ordinary configuration, not something a workflow declares. Declaring a deployment field merely to set one would be a second configuration mechanism for a value that belongs beside the model.

A provider environment variable such as `OLLAMA_MAX_TOKENS` is still read, but as a *fallback*, not an override: it supplies a setting nobody configured, and a configured one wins. The order is

model configuration → environment variable → built-in default

so a value written beside the model cannot be changed by a process-wide variable that says nothing about which model it meant.

5.2 Asking a person

A `@human` action stops and waits. Where the question appears depends on configuration:

An answer is required by default. The only non-blocking form is an explicit `kind="ack", required=False` acknowledgement. It is recorded and resolves to true without creating a human task; confirmations, edits, selections, and inputs cannot use `required=False`.

nothing configured	the terminal running the workflow
a Telegram connector, assigned	a message with buttons

The terminal is right while developing and wrong for a deployment nobody is watching. Provider setup, destination configuration, and assignment do three different things:

```
zg provider configure approval-bot telegram
zg provider set-credential approval-bot
zg connector configure approval-chat approval-bot
zg connector assign Mailbox approval-chat
```

The provider commands name a bot and save its token. The connector configuration says *which* chat. The assignment says *whose action* is routed there: a participant, or one action with `Mailbox.approve_reply`.

The bot token is typed without echo and stays on this machine. The chat id goes in `zippergen.toml` and is committed: a colleague gets the chat, and supplies their own token.

5.3 Reading and writing external services

A workflow declares what it needs, without saying which one:

```
zippergen_connectors = (
    ConnectorRequirement(
        name="call-mailbox", kind="gmail",
        participant="Mailbox", access="read-write",
        capabilities=("read-messages", "mark-processed"),
    ),
)
```

This is a *requirement*: a labelled hole. It says a Gmail mailbox is needed, that `Mailbox` is the participant that uses it, and what will be done with it. It names no account, no address and no credential, so it stays true on every machine. Workflow code plugs into it by name, with `@effect(connector="call-mailbox", ...)`, and never mentions a mailbox address.

Three separate things are involved, and keeping them apart is what makes a workflow portable:

	says	lives in
requirement	what kind of thing is needed	`workflow.py`, committed
deployment field	what a person must be asked	`workflow.py`, committed
configuration	which actual mailbox, chat, or answer	`zippergen.toml`, committed
credential	the token	this machine only, never committed

Human actions need no requirement, because the workflow already gives it away: a `@human` action means a person must answer, and the participant running it says who. An external service is different — nothing in the code says it needs Gmail with permission to mark mail read, unless it is written down. Writing it down is what lets `zg connector check` report an empty hole before a deployment starts, and what tells you exactly which Google permission to grant.

`GmailMailbox.fetch_one_unread()` returns one canonical `gmail_id`, the RFC `message_id`, Gmail's integer `internal_date_ms`, and the raw sender-supplied `date` header. Use the internal date for Gmail inbox ordering; the mail header is not a trusted arrival timestamp.

The project then says which one, assigning a configuration to that requirement by name:

```
zg provider configure google-work google
zg connector configure inbox google-work gmail \
    --query "is:unread in:inbox"
zg connector assign call-mailbox inbox
zg connector configure records google-work google-sheets \
    --spreadsheet-id 1AbC... --tab Calls
zg connector assign call-records records
```

5.4 Authorizing Google

```
zg provider authorize google-work
```

The scopes are not typed. Each requirement declares its kind and its access, so the narrowest set that covers them is already known, and it is printed before the browser opens:

```
Scopes this workflow asks for: gmail.modify, spreadsheets
```

Pass `--scopes` only to authorize outside a project, or to grant something the workflow does not ask for.

This opens a browser and saves the credential here, together with the scopes Google actually granted. Run inside the project and there is nothing further to do.

A server usually has no browser, so it is authorized from a computer that has one:

```
zg provider authorize google-work --handoff   # on your laptop
zg provider accept google-work                # on the server
```

`--handoff` prints an encoded result instead of saving it, and `accept` asks for that result without echo. The credential never appears in a command line, so it does not reach shell history.

Scopes are checked before deployment

Deploying refuses when the granted scopes do not cover what the workflow needs For example, a workflow that marks mail as read needs `gmail.modify`, not `gmail.readonly`. Re-run `authorize` with the missing scopes.

6 Deployment

6.1 Deploy

```
zg deploy
```

The ordinary command writes the bundle, prepares the managed Python environment and service files, checks real dependencies, and starts the service. A failure stops it before starting. Use `--no-start` only when you deliberately want preparation and review to stop before a later `zg deploy start`; it is not a required preliminary step.

To redeploy current source, stop the running service before replacing its bundle and managed environment:

```
zg deploy stop
zg deploy                    # rebuild and start the updated deployment
```

6.2 Configuration: what a deployment is asked

A workflow can declare the values a person must supply for it to be deployed — a mailbox address, a polling interval, whether replies are drafted or sent. The declaration lives with the workflow, so the question, its default, and its permitted answers are reviewed like any other code:

```
zippergen_deployment = DeploymentSpec(
    description="Watch a mailbox and reply.",
    fields=(
        DeploymentField("recipient", "Intake address", target="option",
                        required=True),
        DeploymentField("send_mode", "Reply mode", target="option",
                        default="draft", choices=("draft", "send", "log")),
    ),
)
```

`target` says where an answer is delivered: `"option"` to `--option`, `"input"` to a workflow input, `"env"` to an environment variable. An `env` field may also be `secret`.

Where the answers are. One rule covers every one of them:

> Every non-secret answer is kept in `zippergen.toml`, under `[configuration]`. The deployment profile is derived from it, never authored.

Because the answers belong to the project rather than to one command, both commands use them. `zg run` takes a stored answer as its default and records a new one, so a value given once need not be given again; `--input` still overrides for a single run, and that answer is remembered too. `zg config` lists them beside every other project choice:

```
Configuration
Field       Value              Asks
recipient   alice@example.org  Intake address
send_mode   draft              Reply mode
```

`zg deploy status` does not repeat that table. What a deployment adds is whether the running service still matches those answers, which its freshness checks report.

So the question "I answered that during `zg deploy` — where is it tomorrow?" has a single answer, and it is a file you can open, edit, and commit. Secrets are the one exception: they go to a private file and are never written into project configuration.

Answer a question, or change one later, without building anything:

```
zg config --set recipient=intake@example.org --set send_mode=send
zg config --unset send_mode
```

A value outside a field's declared `choices` is refused, and a name the workflow never declared is refused with the ones it did. Setting and forgetting one field in a single command is refused rather than resolved silently, and `--json` reports what was written rather than falling back to prose. The same answers can also be given while deploying, with `zg deploy --set field=value`, which is convenient when you are deploying anyway.

What `--yes` uses. `--yes` answers every prompt without asking, which is only safe if you can see what it chose. So every deploy prints its configuration and where each value came from:

```
Configuration
Field       Value                From
recipient   intake@example.org   zippergen.toml
send_mode   send                 --set
poll_secs   60                   declared default
```

The order is fixed: `zippergen.toml` first, then the environment for an `env` field, then the declared default, and `--set` overrides all of them. Each `zg deploy` prints this table before publishing the service. For an existing deployment, use `zg config` to inspect the values and `zg deploy status` to see whether the published answers are still current.

Because the project file is where answers are authored and the profile is what is running, the two can differ the moment somebody edits one. So freshness is reported for all three things a deployment is made of:

```
OK   ZipperGen runtime: current (source hash)
OK   workflow source: current (source hash)
WARN configuration: max_rounds: deployed 9, project 2. Redeploy to apply
     the project's answers
```

Secrets are never named there: they are not project configuration, so there is nothing to compare.

6.3 The five lifecycle operations

A project has exactly one deployment. Once initialized, the project id owns one stable deployment address recorded in local checkout state. That is why none of these commands takes a name and why moving the project preserves its deployment.

Two independent questions decide which one you want: does it rebuild what will run, and does it keep the durable state? Nothing else separates them.

	rebuilds	durable state	
`zg deploy`	**yes**	kept	prepare or re-prepare, then start; requires a stopped service
`zg deploy start`	no	kept	start what is already prepared
`zg deploy stop`	no	kept	stop the service; `start` resumes where it was
`zg deploy reset`	no	**emptied**	archive the state and leave the service stopped
`zg deploy remove`	—	archived	unregister the service and take the deployment away

`start` on a deployment that is already running does nothing, so it is safe to repeat. To pick up an edit, use `stop` then `deploy`; ZipperGen refuses `zg deploy` while the service runs so that this choice is explicit rather than silent.

One pair is worth reading twice:

- `reset` is the destructive one, despite sounding milder than `remove`. It keeps the deployment and throws the durable state away. `remove` keeps that state, archived under `ZIPPERGEN_HOME/trash/deployments/`, and only `remove --purge` leaves nothing.

`reset` always leaves the service stopped, whether or not it was running before. It is a state operation, and it has to stop the service to replace the store, so it does not decide on your behalf that the service should come back. The pause matters: a reset clears the connector progress too, so the next start may re-read a mailbox that was already handled. Start it again with `zg deploy start` once you have looked.

Why `run` has no `remove`

`reset` is about the state. `remove` is about the deployment.
A durable run is only a store file, so it has state and nothing else, and `zg run reset` is its whole vocabulary. A deployment is that store plus a registered service, a profile, credentials, a managed environment and a source bundle, so it needs the second verb as well. Both reset commands abandon the current durable execution, but they retain it differently: run reset deletes its record and store unless `--archive` is requested, while deployment reset always archives the store and leaves the service stopped.
Neither substitutes for the other. `remove` deletes credentials, so removing and deploying again costs you a token re-entry rather than being a heavier reset; and without `remove` there is no way to unregister the service, so deleting a project directory would leave the supervisor pointing at a bundle that no longer exists.

Stopping is not losing

Durable state is the current state of the computation, committed after every step. `stop` therefore keeps everything that finished: variables, control position, outstanding messages, and any human task still waiting for an answer. `start` carries on from there.
The one step in flight is interrupted. If a model call or an `@effect` had not recorded its result, it runs again on the next start, side effect included. That is the same at-least-once boundary a crash has, and it is the only work `stop` can cost you.

Which edits a running deployment can resume through

Durable state records *positions in the projected program* — role `Extractor` is at this node, holding these values. If the program changes shape, a stored position means something else, so ZipperGen checks once at startup and refuses rather than resuming into the wrong place:

```
The workflow changed since this durable state was written, so the stored
control positions no longer mean the same thing. Reset the deployment with
'zg deploy reset' to start fresh, or restore the previous version of the
workflow to resume it.
```

The split is simple: what a step *does* may change freely; where the steps *are* may not.

edit	resumes
an action's Python body	yes
an LLM prompt	yes
what a guard computes	yes
the model, connector, chat or spreadsheet	yes
a message added, removed or moved	no
a participant added or removed	no
a renamed or retyped value a step reads or writes	no

So fixing a prompt and redeploying keeps everything in flight; restructuring the protocol asks you to choose. `docs/durable-storage.md` states the rule exactly.

6.4 Operating one

`zg deploy status`	is it running, what state does it hold, which action is live, what failed last, and are its runtime and workflow bundle current
`zg deploy logs`	what it has been doing
`zg deploy inspect`	where each participant's local program is now
`zg deploy inspect --watch`	keep that position open and refresh it in place
`zg deploy trace`	recent protocol events, prefaced by service and runtime freshness
`zg deploy trace --follow`	append newly committed events without redrawing the table
`zg deploy tasks`	decisions waiting for a person
`zg deploy approve --task ID`	answer one locally; use `--token-stdin` for an external capability token
`zg deploy check`	readiness checks, including private file modes; `--repair-permissions` fixes managed paths
`zg deploy start` / `zg deploy stop`	move the service
`zg deploy compact`	trim optional history and rotate logs; refuses while running
`zg deploy reset --yes`	archive durable state; start again yourself
`zg deploy remove`	unregister it; archive profile, store and log unless purged
`zg deploy list`	every deployment on this computer, orphans included
`zg deploy prune`	clear orphans and stale archives

`inspect --watch` redraws current state in place. Trace is an event record: `trace --follow`

preserves its table and appends rows. A newly committed external action appears as *start*, followed later by *done* or *failed*. In a static trace, an unmatched start is called *incomplete*: no terminal event was recorded, which does not prove whether the remote operation succeeded. Use status for current live activity.

6.5 How much trace it keeps

What recovery reads does not grow with execution history: one row per participant, plus messages nobody has absorbed yet. The trace is separate, and recovery never reads it, so how much of it to keep is your choice. A store keeps the newest 10,000 events unless told otherwise.

That is a FIFO count of events, not a time window or a size. A workflow whose events carry large values — a whole email, a long model answer — holds far more bytes at the same budget than one passing short strings. Frequent routine events also evict old incidents at exactly the same rate as important ones. A measured four-participant poller writing 14 events per minute filled the default in about 12 hours even while nearly every poll found nothing.

Measure the real event rate and payload distribution before choosing a value. A very large budget is not free: periodic pruning uses an offset scan on the writer path, and the SQLite pages and WAL mean a row count is not an exact on-disk ceiling.

```
zg deploy --history-keep 50000              # a wider window
zg deploy --history-keep 0                  # record no trace at all
zg deploy compact --set-history-keep 2000   # change it later, and apply it now
zg run --durable --history-keep 500         # the same choice for one run
```

`zg deploy status` and `zg run status` report the budget and how much of it is in use. A bare `zg deploy compact` trims to that budget; it does not empty the store. Event numbers keep climbing when the budget changes, so `trace --after N` stays correct.

6.6 What removal archives

durable store, log	archived	the record of what actually ran
profile	archived	what produced them
secrets	**deleted**	must not be left behind
environment, bundle	deleted	rebuilt by deploying again
service files	deleted	regenerated

Archived means moved to ZipperGen's trash directory, not left where it was: after a removal the deployment no longer exists where the commands look for it, and a later `zg deploy` builds a new one starting from an empty store. `--purge` keeps nothing, including the store. Both commands name what they are about to destroy before asking, so read the list rather than the verb.

What survives lands under `ZIPPERGEN_HOME/trash/`, readable only by you, in three places: `deployments/` for removals, `deployment-stores/` for resets, and `deployment-logs/` for rotated logs.

```
zg deploy prune                         # orphaned deployments, and archives over 30
    days old
zg deploy prune --keep-days 7           # a shorter undo window
```

Pruning is decided by age, never by size, because the archive exists so that a mistaken `remove` can be undone. Today's archive is exactly the one somebody may still need, so it is never the one deleted.

6.7 Several deployments, one bot

Sharing one Telegram bot across projects is expected and supported. It needs one thing you cannot see, so it is worth knowing why.

A Telegram connector also names the user allowed to answer. For a private chat this defaults to the chat id. A group must set it explicitly:

```
zg connector configure approval-chat approval-bot \
  --chat-id=-100123456 --allowed-user-id=12345678
```

Other members can see the buttons but cannot settle the durable task.

Telegram's inbound queue is *one queue per bot with one cursor*, and reading it destroys: an update is confirmed, and forgotten, as soon as a later read supplies a higher offset. Two deployments reading independently would therefore confirm each other's approvals out of existence. Sending has no such problem; it is an ordinary request.

So the cursor belongs to the bot, not to a deployment:

`ZIPPERGEN_HOME/connectors/telegram-<id>.sqlite`	the shared cursor, and updates not yet absorbed
`ZIPPERGEN_HOME/connectors/telegram-<id>.lock`	ensures exactly one reader at a time

Whichever deployment takes the lock reads Telegram on everyone's behalf and writes what it finds into the shared inbox. Every deployment, lock-holder or not, then takes the updates its own tasks issued. Losing the race costs you nothing: you still receive your own approvals, you simply did not do the fetching.

Two consequences follow. Deployments sharing a bot must run on the same machine, because the lock is a local file. And an update addressed to a deployment that is currently stopped waits rather than being discarded, so `zg deploy prune` clears only ones older than the retention window.

Why the lock file is never deleted

The lock lives on the inode, not the path. Removing the file while another process holds it would let the next process lock a fresh inode and believe it had exclusive access, which is precisely the guarantee this rests on. The file is empty and permanent; nothing needs to clean it up.

6.8 Moving a project to a server

1. Clone the repository. The workflow, specification and `zippergen.toml` come with it.
2. `zg validate` confirms the module loads in that environment.
3. Install what the workflow needs. Google connectors require the optional extra, `zippergen[google]`, in the environment that runs `zg`; `zg provider check` reports it as *google support installed*.
4. Supply what the machine needs. Credentials belong to the provider connection, not to the model or connector that uses it: model keys in the environment or via `zg provider set-credential`, the Telegram token the same way, and Google with `zg provider authorize --handoff` on your laptop and `zg provider accept` there.
5. `zg deploy` prepares, checks, enables and starts the service.

Nothing secret is in the repository, so step 3 is the whole difference between a clone and a running service.

On Linux, bare deploy enables the systemd user unit, but the account's user manager must also survive logout and start at boot. If server policy permits it, enable linger once and inspect both facts:

```
loginctl enable-linger "$USER"
zg deploy check
```

The check reports *systemd autostart* and *systemd linger* separately. Before relying on the service unattended, log out once and reboot once, then confirm with `zg deploy status` that it came back.

7 Working with a coding agent

ZipperGen ships instructions for coding agents. `zg init` writes an `AGENTS.md` pointing at them and a `CLAUDE.md` that imports those same instructions for Claude Code. `zg skill` prints the full guidance.

An already open coding-agent conversation does not automatically reload that guidance after ZipperGen is upgraded. Start a new session, or explicitly ask the agent to run `zg skill` again. An agent sandbox may also use a different `ZIPPERGEN_HOME`; `zg config` shows the active private state location, and credential readiness should be confirmed in the user's own terminal when the sandbox cannot read it.

```
zg skill                  # what an agent should follow
zg skill --agents-md      # a pointer file for an existing project
```

A reasonable division:

the agent	writes and changes the workflow and specification, validates, runs, inspects projections, prepares deployments
you	credentials, authorization, and starting production

That division is a convention, not a wall: an agent with a shell can run any command you can, and can read any file you can. Credentials are typed rather than passed as arguments, so they stay out of the agent's conversation, your shell history and the repository. But once saved they live in a file under `ZIPPERGEN_HOME` that your user can read.

That distinction is worth keeping straight. Entering a credential interactively protects it *at the moment of entry*. It is not a capability boundary. For a real one, run the agent under a different account, or in an environment that does not carry `ZIPPERGEN_HOME`. Nothing in ZipperGen can supply that, because a process running as you can do what you can do.

The specification is yours and the agent's

ZipperGen does not check that `workflow.py` implements `specification.md`, because it cannot: one is prose. There is no gate, no provenance, and no ceremony around it. Keep it accurate the way you keep a README accurate, and tell your agent to update it before changing behaviour, which the shipped skill does.

8 When something is wrong

8.1 The workflow will not load

`zg validate` reports the exception. Common causes: a missing import, an action used before it is defined, or a compound condition without parentheses (`if (not agreed) @ LLM1:`).

8.2 A participant seems to be waiting forever

Look at what it actually runs:

```
zg show --agent ThatParticipant
```

A projected program cannot wait for a message nobody sends, so if a participant is stuck the cause is usually an action that never returns, such as a model call without a timeout, or a human action nobody has answered. Use `zg run tasks` for a durable run or `zg deploy tasks` for the deployment.

8.3 Only one branch ever runs

You are almost certainly using `--llm mock`, which answers every action identically. Script the responses instead.

8.4 A deployment refuses to start

Starting probes every model, assistant CLI, and connector. The message names what failed. The usual causes are a missing API key on that machine, a Google authorization that does not cover the workflow's scopes, or a Telegram token that was never supplied there.

8.5 A run cannot be resumed or inspected

A plain run keeps nothing. Use `--durable` for a run you want to come back to.

8.6 Connector configured but the question still appears in the terminal

Configuring says which chat. Assigning says who is asked there. Both are needed:

```
zg connector assign Mailbox approval-chat
```

9 Command reference

Project	
`init [NAME]`	create a project here
`skill`	print the coding-agent instructions
Inspect	
`validate`	load, project, and check
`show`	render the protocol or one participant
`snapshot` / `diff`	record and compare semantics
Run	
`run`	execute, with `--durable` and `--resume`
`run status/reset/inspect/trace/tasks/approve`	manage, observe, or answer a durable run

Configure	
`config` / `check`	show offline configuration or check all live readiness
`provider configure NAME KIND`	save one named provider identity
`provider set-credential NAME`	save its key or bot token privately
`provider authorize/accept NAME`	move Google authorization from a browser computer
`model configure NAME CONNECTION MODEL`	save one named model configuration
`model assign TARGET NAME`	route a participant or action
`model unassign TARGET` / `model remove NAME`	remove routing or an unused configuration
`assistant configure NAME BACKEND`	save Codex or Claude under a project name
`assistant assign TARGET NAME`	route an assistant participant or action
`assistant check [NAME]`	verify the selected CLI and required safety options
`assistant unassign TARGET` / `assistant remove NAME`	remove routing or an unused configuration
`connector configure NAME CONNECTION [KIND]`	save one named destination/resource; infer the kind when unique
`connector assign TARGET NAME`	fill a declared requirement, or route human actions
`TARGET` ∈ {`default`, participant, `Participant.action`}	three levels, most specific wins; same for models and assistants
`connector unassign TARGET`	remove connector routing
`TYPE rename OLD NEW`	rename one configuration and everything naming it
`connector check [NAME]` / `connector remove NAME`	check or remove
`completion zsh|bash|fish`	print dynamic shell completion

Deploy	
`deploy [--no-start]`	prepare, check, and normally start
`deploy list/prune`	find deployments and archive orphans
`deploy start/stop`	move the service
`deploy status/logs/check`	operate and diagnose
`deploy inspect/trace/tasks/approve`	observe or answer the deployment
`deploy compact/reset/remove`	trim history and logs, reset state, or unregister and archive

Every command takes `--help`. Commands that act on a workflow default to the project's. Commands that act on a deployment default to the most recent.

`zippergen` and `zg` are the same program.

10 Where the guarantee comes from

Projection is syntax-directed. For each decision, every participant is one of three things:

decider	evaluates the condition and broadcasts the outcome
recipient	appears in a branch, receives the outcome and follows it
bystander	appears in neither branch, the decision is erased from its program

The `Writer` in the tutorial is a bystander. Its local program does not mention the approval, so it cannot wait on it or disagree about it.

The main theorem states that the projected local programs produce exactly the behaviours of the global workflow, up to the control messages projection introduces. Deadlock freedom follows for well-formed workflows. Both are machine-checked in Lean 4.

That is the trade the whole system makes: write one protocol, accept the language's restrictions, and get a distributed implementation that cannot deadlock, rather than writing each agent separately and hoping.